# Assignment 2: Conditional Rendering and List

**Subject: ReactJS | Unit 2: Conditional Rendering and List**

B.Sc. Computer Science

## Q1. Conditional Rendering using if-else Statement

if-else is the most basic way to conditionally render content in React. We use a regular JavaScript if-else inside the render() method (class) or inside the function body to decide which JSX to return.

**Program / Code:**

```
import React, { Component } from 'react';

class IfElseDemo extends Component {
  constructor(props) {
    super(props);
    this.state = { isAdult: true };
  }

  render() {
    let message;
    if (this.state.isAdult) {
      message = <h3>You are an Adult. Access Granted!</h3>;
    } else {
      message = <h3>You are a Minor. Access Denied!</h3>;
    }
    return (
      <div>
        <h2>if-else Conditional Rendering</h2>
        {message}
      </div>
    );
  }
}

export default IfElseDemo;
```

**Expected Output:**

```
Output (isAdult = true):
  if-else Conditional Rendering
  You are an Adult. Access Granted!

Output (isAdult = false):
  You are a Minor. Access Denied!
```

## Q2. Login / Logout Button using Conditional Rendering

A common use case of conditional rendering is toggling UI elements. Here we conditionally show a Login or Logout button depending on the isLoggedIn state. Clicking either button toggles the state.

**Program / Code:**

```
import React, { useState } from 'react';

function LoginLogout() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);

  const handleToggle = () => {
    setIsLoggedIn(!isLoggedIn);
  };

  return (
    <div>
      <h2>Login / Logout Demo</h2>
      <p>Status: {isLoggedIn ? 'Logged In' : 'Logged Out'}</p>
      {isLoggedIn ? (
        <button onClick={handleToggle}>Logout</button>
      ) : (
        <button onClick={handleToggle}>Login</button>
      )}
    </div>
  );
}

export default LoginLogout;
```

**Expected Output:**

```
Output (initial - not logged in):
  Status: Logged Out
  [ Login ]

Output (after clicking Login):
  Status: Logged In
  [ Logout ]
```

## Q3. Logical AND (&&) Operator for Conditional Rendering

The && (logical AND) operator is a concise way to render content conditionally. If the left side is true, React renders the right side. If false, React renders nothing (ignores false value in JSX).

**Program / Code:**

```
import React, { useState } from 'react';

function AndOperator() {
  const [hasNotification, setHasNotification] = useState(true);
  const [isPremium, setIsPremium]             = useState(false);

  return (
    <div>
      <h2>Logical AND (&&) Demo</h2>

      {/* Renders only if hasNotification is true */}
      {hasNotification && (
        <p style={{color:'red'}}>You have a new notification!</p>
      )}

      {/* Does NOT render because isPremium is false */}
      {isPremium && (
        <p style={{color:'gold'}}>Welcome Premium Member!</p>
      )}

      <p>isPremium is false so premium message is hidden.</p>
    </div>
  );
}

export default AndOperator;
```

**Expected Output:**

```
Output:
  Logical AND (&&) Demo
  You have a new notification!    <- shown (true && ...)
                                  <- hidden (false && ...)
  isPremium is false so premium message is hidden.
```

---

## Q4. Show Message if User is Logged In, Otherwise Hide It

Using the ternary operator (condition ? a : b) we can show one element when a condition is true and a different one (or nothing) when false. Returning null from JSX renders nothing at all.

**Program / Code:**

```
import React, { useState } from 'react';

function WelcomeMessage() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);

  return (
    <div>
      <h2>Welcome Message Demo</h2>

      {isLoggedIn ? (
        <div style={{background:'#e8f5e9', padding:'10px'}}>
          <h3>Welcome Back, Rahul!</h3>
          <p>You are successfully logged in.</p>
        </div>
      ) : (
        <p style={{color:'red'}}>Please log in to see the message.</p>
      )}

      <button onClick={() => setIsLoggedIn(!isLoggedIn)}>
        {isLoggedIn ? 'Logout' : 'Login'}
      </button>
    </div>
  );
}

export default WelcomeMessage;
```

**Expected Output:**

```
Output (logged out):
  Please log in to see the message.   [ Login ]

Output (logged in):
  Welcome Back, Rahul!
  You are successfully logged in.     [ Logout ]
```

## Q5. Switch Case Statement – Display Day of Week

Switch-case works well when there are multiple possible values. We use a switch inside a function that returns different JSX for each day. This is cleaner than writing multiple if-else blocks.

**Program / Code:**

```
import React, { useState } from 'react';

function DayOfWeek() {
  const [day, setDay] = useState(1);

  function getDayName(d) {
    switch(d) {
      case 1: return 'Monday - Start of the week!';
      case 2: return 'Tuesday - Keep going!';
      case 3: return 'Wednesday - Midweek!';
      case 4: return 'Thursday - Almost there!';
      case 5: return 'Friday - Last working day!';
      case 6: return 'Saturday - Weekend starts!';
      case 7: return 'Sunday - Rest day!';
      default: return 'Invalid day number!';
    }
  }

  return (
    <div>
      <h2>Day of Week using Switch Case</h2>
      <p>Day {day}: <b>{getDayName(day)}</b></p>
      <input type='number' min='1' max='7' value={day}
             onChange={(e) => setDay(Number(e.target.value))} />
    </div>
  );
}

export default DayOfWeek;
```

**Expected Output:**

```
Output (day = 1): Day 1: Monday - Start of the week!
Output (day = 5): Day 5: Friday - Last working day!
Output (day = 7): Day 7: Sunday - Rest day!
Output (day = 9): Invalid day number!
```

---

## Q6. Prevent Component from Rendering using null Return

In React, if a component returns null, it renders nothing. This is useful to completely hide a component based on a condition. The component still exists in the tree but produces no DOM output.

**Program / Code:**

```
import React, { useState } from 'react';

// This component renders nothing when show is false
function WarningBanner({ show }) {
  if (!show) {
    return null;  // Renders nothing
  }
  return (
    <div style={{background:'#fff3cd', padding:'10px', border:'1px solid #ffc107'}}>
      <b>Warning:</b> You have unsaved changes!
    </div>
  );
}

function App() {
  const [showWarning, setShowWarning] = useState(true);
  return (
    <div>
      <h2>Prevent Rendering using null</h2>
      <WarningBanner show={showWarning} />
      <button onClick={() => setShowWarning(!showWarning)}>
        {showWarning ? 'Hide Warning' : 'Show Warning'}
      </button>
    </div>
  );
}

export default App;
```

**Expected Output:**

```
Output (show=true):  Warning: You have unsaved changes!  [ Hide Warning ]
Output (show=false): (nothing shown)                      [ Show Warning ]
```

## Q7. List of Student Names using map() Function

The JavaScript map() function iterates over an array and returns a new array of JSX elements. React renders this array as a list. Each item must have a unique key prop for efficient DOM updates.

**Program / Code:**

```
import React from 'react';

function StudentList() {
  const students = [
    'Rahul Patil',
    'Priya Sharma',
    'Amit Desai',
    'Sneha Kulkarni',
    'Rohan Jadhav'
  ];

  return (
    <div>
      <h2>Student Names List</h2>
      <ul>
        {students.map((name, index) => (
          <li key={index}>
            {index + 1}. {name}
          </li>
        ))}
      </ul>
      <p>Total Students: {students.length}</p>
    </div>
  );
}

export default StudentList;
```

**Expected Output:**

```
Output:
  Student Names List
  1. Rahul Patil
  2. Priya Sharma
  3. Amit Desai
  4. Sneha Kulkarni
  5. Rohan Jadhav
  Total Students: 5
```

---

## Q8. Render List of Products with Unique Keys

When rendering lists of objects, use a unique id field as the key prop. Using array index as key is acceptable for static lists but using a unique id is the best practice for dynamic lists.

**Program / Code:**

```
import React from 'react';

function ProductList() {
  const products = [
    { id: 101, name: 'Laptop',     price: 55000, category: 'Electronics' },
    { id: 102, name: 'Mobile',     price: 15000, category: 'Electronics' },
    { id: 103, name: 'Headphones', price:  2500, category: 'Accessories' },
    { id: 104, name: 'Mouse',      price:   800, category: 'Accessories' },
  ];

  return (
    <div>
      <h2>Product List</h2>
      <table border='1' cellPadding='8'>
        <thead>
          <tr><th>ID</th><th>Name</th><th>Price</th><th>Category</th></tr>
        </thead>
        <tbody>
          {products.map(p => (
            <tr key={p.id}>
              <td>{p.id}</td><td>{p.name}</td>
              <td>Rs. {p.price}</td><td>{p.category}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default ProductList;
```

**Expected Output:**

```
Output:
  ID  | Name       | Price     | Category
  101 | Laptop     | Rs. 55000 | Electronics
  102 | Mobile     | Rs. 15000 | Electronics
  103 | Headphones | Rs.  2500 | Accessories
  104 | Mouse      | Rs.   800 | Accessories
```

## Q9. React key Prop in Lists

The key prop is a special attribute that helps React identify which items have changed, been added, or removed. Keys must be unique among siblings. Below we demonstrate the difference between using index vs unique id as key.

**Program / Code:**

```
import React, { useState } from 'react';

function KeyPropDemo() {
  const [items, setItems] = useState([
    { id: 1, text: 'Learn React' },
    { id: 2, text: 'Understand JSX' },
    { id: 3, text: 'Practice Props' },
    { id: 4, text: 'Master State' },
  ]);

  const removeFirst = () => {
    setItems(items.slice(1));  // Remove first item
  };

  return (
    <div>
      <h2>React key Prop Demo</h2>
      <ul>
        {items.map(item => (
          // Using unique id as key (Best Practice)
          <li key={item.id}>{item.id}. {item.text}</li>
        ))}
      </ul>
      <button onClick={removeFirst}>Remove First Item</button>
    </div>
  );
}

export default KeyPropDemo;
```

**Expected Output:**

```
Output (initial):
  1. Learn React
  2. Understand JSX
  3. Practice Props
  4. Master State
  [ Remove First Item ]
Output (after click): items 2, 3, 4 remain
```

---

## Q10. Iterate Array and Display Elements using map()

map() is the standard React way to render arrays. Here we demonstrate iterating different types of arrays: numbers, strings, and objects — and displaying them with styled output using JSX.

**Program / Code:**

```
import React from 'react';

function ArrayIterator() {
  const numbers = [10, 25, 37, 42, 58, 63];
  const colors  = ['Red', 'Green', 'Blue', 'Yellow', 'Orange'];
  const marks   = [
    { subject: 'ReactJS',    score: 88 },
    { subject: 'Node.js',    score: 75 },
    { subject: 'MongoDB',    score: 91 },
  ];

  return (
    <div>
      <h2>Array Iteration using map()</h2>

      <h3>Numbers:</h3>
      {numbers.map((n, i) => <span key={i}>{n}  </span>)}

      <h3>Colors:</h3>
      <ul>{colors.map((c, i) => <li key={i}>{c}</li>)}</ul>

      <h3>Marks:</h3>
      {marks.map((m, i) => (
        <p key={i}>{m.subject}: {m.score}/100</p>
      ))}
    </div>
  );
}

export default ArrayIterator;
```

**Expected Output:**

```
Output:
  Numbers: 10  25  37  42  58  63
  Colors:  Red | Green | Blue | Yellow | Orange
  Marks:
    ReactJS: 88/100
    Node.js: 75/100
    MongoDB: 91/100
```

## Q11. Create and Access Refs using useRef Hook

useRef() creates a mutable ref object whose .current property holds a reference to a DOM element or value. Unlike state, changing a ref does NOT trigger a re-render. Refs are used to access DOM nodes directly.

**Program / Code:**

```
import React, { useRef } from 'react';

function RefDemo() {
  // Create a ref
  const inputRef    = useRef(null);
  const headingRef  = useRef(null);

  const showValue = () => {
    // Access ref's current DOM element
    alert('Input Value: ' + inputRef.current.value);
  };

  const changeColor = () => {
    headingRef.current.style.color = 'blue';
  };

  return (
    <div>
      <h2 ref={headingRef}>useRef Hook Demo</h2>
      <input ref={inputRef} type='text' placeholder='Type here...' />
      <button onClick={showValue}>Show Value</button>
      <button onClick={changeColor}>Change Heading Color</button>
    </div>
  );
}

export default RefDemo;
```

**Expected Output:**

```
Output:
  [ useRef Hook Demo ]  <- heading (turns blue on button click)
  [ Type here... ] [ Show Value ] [ Change Heading Color ]
  Clicking 'Show Value' alerts: Input Value: (typed text)
  Clicking 'Change Heading Color' turns heading text blue
```

---

## Q12. Focus an Input Field using React Refs

One of the most common uses of refs is programmatically focusing an input field. We call inputRef.current.focus() on a button click, which focuses the input without any user clicking on it directly.

**Program / Code:**

```
import React, { useRef } from 'react';

function FocusInput() {
  const inputRef = useRef(null);

  const handleFocus = () => {
    inputRef.current.focus();          // Focus the input
  };

  const handleClear = () => {
    inputRef.current.value = '';       // Clear the input
    inputRef.current.focus();          // Then focus it
  };

  return (
    <div>
      <h2>Focus Input using Refs</h2>
      <input
        ref={inputRef}
        type='text'
        placeholder='Click button to focus me'
        style={{ padding: '8px', width: '250px' }}
      />
      <br /><br />
      <button onClick={handleFocus}>Focus Input</button>
      <button onClick={handleClear}>Clear & Focus</button>
    </div>
  );
}

export default FocusInput;
```

**Expected Output:**

```
Output:
  Focus Input using Refs
  [ Click button to focus me ]
  [ Focus Input ]  [ Clear & Focus ]
  Clicking 'Focus Input' -> cursor appears in the text box
  Clicking 'Clear & Focus' -> clears text and focuses input
```

## Q13. Event Binding using bind() Method

In class components, event handler methods do not automatically bind 'this'. We must explicitly bind them. Using bind() in the constructor is the recommended approach as it binds only once per instance.

**Program / Code:**

```
import React, { Component } from 'react';

class BindDemo extends Component {
  constructor(props) {
    super(props);
    this.state = { message: 'Click the button!' };

    // Binding in constructor (Best Practice)
    this.handleClick = this.handleClick.bind(this);
  }

  handleClick() {
    this.setState({ message: 'Button Clicked! bind() works!' });
  }

  handleHover() {
    // Inline bind in JSX (not recommended for performance)
    this.setState({ message: 'Mouse is hovering!' });
  }

  render() {
    return (
      <div>
        <h2>Event Binding using bind()</h2>
        <p>{this.state.message}</p>
        <button onClick={this.handleClick}>
          Click Me (constructor bind)
        </button>
        <button onMouseEnter={this.handleHover.bind(this)}>
          Hover Me (inline bind)
        </button>
      </div>
    );
  }
}

export default BindDemo;
```

**Expected Output:**

```
Output (initial):      Click the button!
Output (after click):  Button Clicked! bind() works!
Output (on hover):     Mouse is hovering!
```

---

## Q14. Event Binding using Arrow Function

Arrow functions automatically capture 'this' from the surrounding lexical scope, so they do not need explicit binding. Class fields with arrow functions and inline arrow functions in JSX are two common approaches.

**Program / Code:**

```
import React, { Component } from 'react';

class ArrowBindDemo extends Component {
  state = { count: 0, lastEvent: 'None' };

  // Arrow function as class field (auto-binds this)
  handleIncrement = () => {
    this.setState({
      count: this.state.count + 1,
      lastEvent: 'Increment clicked'
    });
  };

  handleDecrement = () => {
    this.setState({
      count: this.state.count - 1,
      lastEvent: 'Decrement clicked'
    });
  };

  render() {
    return (
      <div>
        <h2>Event Binding using Arrow Functions</h2>
        <h3>Count: {this.state.count}</h3>
        <p>Last Event: {this.state.lastEvent}</p>
        <button onClick={this.handleIncrement}>Increment (+)</button>
        <button onClick={this.handleDecrement}>Decrement (-)</button>
        {/* Inline arrow function */}
        <button onClick={() => this.setState({ count: 0, lastEvent: 'Reset' })}> Reset </button>
      </div>
    );
  }
}

export default ArrowBindDemo;
```

**Expected Output:**

```
Output (initial):         Count: 0   |  Last Event: None
Output (after Increment): Count: 1   |  Last Event: Increment clicked
Output (after Decrement): Count: 0   |  Last Event: Decrement clicked
Output (after Reset):     Count: 0   |  Last Event: Reset
```

## Q15. Manage Component State – Counter Application

This comprehensive counter demonstrates full state management: increment, decrement, reset, and step-based counting. State changes trigger automatic UI re-renders in React.

**Program / Code:**

```
import React, { Component } from 'react';

class CounterApp extends Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0,
      step: 1,
      history: []
    };
  }

  updateCount = (newCount, action) => {
    this.setState(prev => ({
      count: newCount,
      history: [...prev.history, action]
    }));
  };

  render() {
    const { count, step, history } = this.state;
    return (
      <div>
        <h2>Counter State Management App</h2>
        <h1 style={{color: count < 0 ? 'red' : 'green'}}>{count}</h1>
        <div>
          <label>Step: </label>
          <input type='number' value={step} min='1'
            onChange={e => this.setState({step: Number(e.target.value)})}
          />
        </div>
        <button onClick={() => this.updateCount(count + step, `+${step}`)}>
          Increment by {step}
        </button>
        <button onClick={() => this.updateCount(count - step, `-${step}`)}>
          Decrement by {step}
        </button>
        <button onClick={() => this.setState({count:0, history:[]})}>
          Reset
        </button>
        <p>History: {history.join(', ') || 'No actions yet'}</p>
      </div>
    );
  }
}

export default CounterApp;
```

**Expected Output:**

```
Output (initial):     0 (green)  |  Step: 1
                      [ Increment by 1 ] [ Decrement by 1 ] [ Reset ]
                      History: No actions yet
Output (after +1,+1): 2 (green)  |  History: +1, +1
Output (count < 0):   -1 (red)   |  History: +1, +1, -1, -1, -1
```

---

End of Assignment 2 — Conditional Rendering and List Topics covered: if-else, &&, ternary, switch-case, null return, map(), key prop, useRef, event binding (bind & arrow functions), state management.